Precision Routing for Efficient Robot Policy Deployment on Edge Devices

CS 2420 Project Proposal

Brianna Grissom, Hang Le, Shresth Verma, Lester Xue

October 17, 2026

1 Introduction

Robot policies are increasingly built on top of large vision-language models (VLMs). Vision-language-action (VLA) models such as RT-2, OpenVLA, and π_0 fine-tune a pretrained VLM backbone to map camera images and a natural-language instruction directly to robot actions. Inheriting internet-scale visual and semantic knowledge allows a single generalist policy to follow open-ended instructions, recognize novel objects and scenes, and transfer across tasks and embodiments far better than policies trained from scratch on robot data alone.

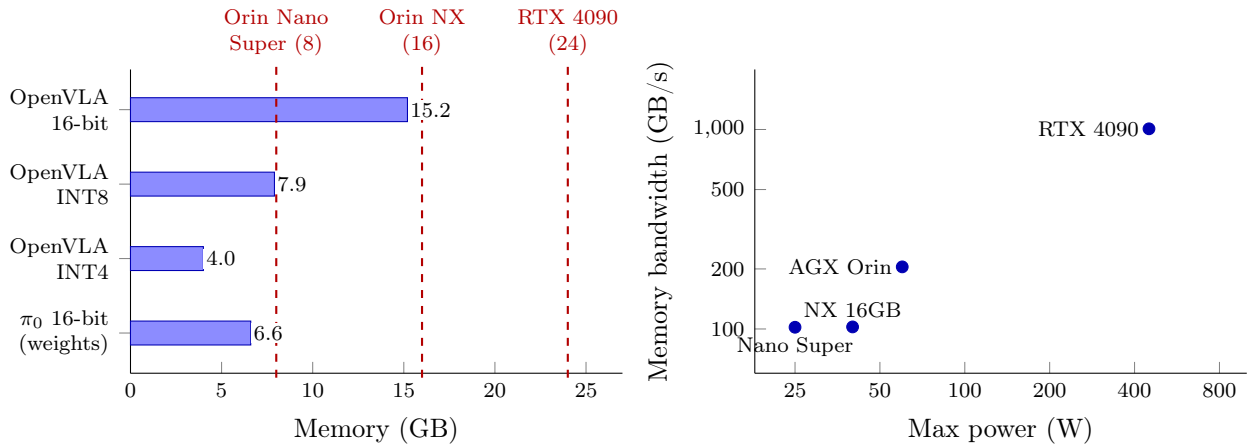
This capability comes at a substantial computational cost. VLA policies impose memory, bandwidth, and power demands that exceed what is available onboard most robots, yet closed-loop control requires low and predictable latency that offloading to a remote server cannot guarantee. OpenVLA requires approximately 15 GB in a 16-bit configuration, nearly double the entire 8 GB shared memory of a Jetson Orin Nano Super. Even the smaller π_0 contains approximately 3.3 billion parameters, requiring about 6.6 GB for its 16-bit weights alone, before accounting for inference buffers, perception, and control software. Even on a desktop RTX 4090, OpenVLA runs at only about 6 queries per second, while edge modules offer 4–10× less memory bandwidth. Figure 1 summarizes this gap between policy requirements and edge hardware.

Prior work has shown that quantization is an effective way to close this gap. On a Jetson AGX Orin, QAIL reports that INT4 weight quantization reduces OpenVLA's memory footprint from 15.2 to 4.0 GB and its inference latency from 955 to 375 ms, an approximately 2.5× speedup with corresponding energy savings. These gains stem from the memory-bound nature of batch-size-one autoregressive decoding, which re-reads every model weight to generate each action token. However, quantization also reduces task accuracy. In the same evaluation, 4-bit AWQ lowers OpenVLA's average LIBERO success from 74.0% to 70.8%, and LIBERO-Long success from 50.8% to 47.2%. Quantization-aware fine-tuning recovers much of this loss (73.1%), but any single fixed precision still commits the policy to one point on the accuracy–efficiency tradeoff for every state it encounters.

This leads to the central question of our project:

> *How can robot policies be deployed efficiently on compute-limited edge devices while achieving both a high success rate and low latency?*

Specifically, we propose learning a *router* that, at each control step, switches among quantized versions of the same policy based on the complexity of the current state. Our hypothesis is that *not all states require equal precision*: moving the arm through free space tolerates coarse actions, whereas



(a) Policy memory vs. device memory.

(b) Bandwidth vs. power budget.

Figure 1: The deployment gap. (a) Memory needed by OpenVLA at different precisions and by π_0's 16-bit weights, against the memory of edge devices (dashed). Jetson memory is shared by CPU and GPU. (b) Edge modules trade memory bandwidth, which bounds batch-size-one decoding, for a lower power budget. Sources: OpenVLA, π_0, QAIL, and NVIDIA specifications; power figures are specified maxima, not measured inference power.

grasping, insertion, and other contact-rich phases demand precise ones. Routing easy states to low-precision models and reserving high precision for critical states should therefore reduce average latency and energy while preserving the success rate of the full-precision policy.

Previous work on dynamic precision for robot policies relies on heuristic routers. DyQ-VLA switches activation precision using a hand-designed motion score with fixed thresholds, and MotionVQ switches between coarse and fine weights based on a fixed motion threshold. Such heuristics must be tuned by hand and use motion as a proxy for how much precision a state actually needs. Instead, we propose a lightweight router learned offline. We label states visited by the policy with the estimated loss in performance from acting at each precision, train a small classifier on cheap features (proprioception, action history, and previous precision choices) to predict the cheapest acceptable precision, and refine it with DAgger-style data aggregation so that training states match those the routed policy visits. Because the router operates on features that are already available, its overhead is negligible relative to the savings from reduced precision. We also consider training the router with reinforcement learning, treating precision selection as a policy, as an extension.

2 Problem Formulation

2.1 Setup

We model robot manipulation as a finite-horizon decision process with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel P, initial-state distribution ρ_0, and horizon T. The input to the policy at time t, which we write s_t, consists of the camera image, proprioception, and the language instruction. An episode $\tau = (s_0, a_0, \ldots, s_T)$ either completes the task or does not; we denote success by the indicator $\mathrm{succ}(\tau) \in \{0, 1\}$. We do not assume access to a dense reward or a learned value function, since VLA policies are trained by imitation rather than reinforcement learning.

Let π_1 be a pretrained full-precision VLA policy. Post-training quantization of π_1 yields a family of frozen policies $\{\pi_k\}_{k=1}^{K}$ at bit-widths $\mathcal{B} = \{b_1 > b_2 > \cdots > b_K\}$, with π_1 serving as the reference.

Each policy has a per-call inference cost $c_k > 0$ (latency or energy measured on the target hardware), with $c_1 > c_2 > \cdots > c_K$.

A router $g_\phi(k \mid z_t)$ selects which policy to query at each step from a cheap feature vector $z_t = f(s_t, a_{t-1}, k_{t-1})$, such as proprioception, recent actions, and the previous precision choice. The routed policy π_ϕ acts as

$$k_t \sim g_\phi(\cdot \mid z_t), \qquad a_t \sim \pi_{k_t}(\cdot \mid s_t). \tag{1}$$

Our goal is to maximize task success while minimizing compute:

$$\max_\phi \; S(\pi_\phi) - \lambda\, C(\pi_\phi), \qquad S(\pi_\phi) = \Pr_{\tau \sim \pi_\phi}\big[\mathrm{succ}(\tau) = 1\big], \qquad C(\pi_\phi) = \mathbb{E}_{\tau \sim \pi_\phi}\Big[\sum_{t=0}^{T-1} c_{k_t}\Big], \tag{2}$$

where $\lambda \geq 0$ trades off success against compute.

2.2 Estimating the Precision Gap

Optimizing Eq. (2) directly would require knowing how a single low-precision decision changes the probability of success, which is unavailable without a critic. Instead, we treat the full-precision policy as the reference behavior: a state tolerates low precision if the quantized policy behaves like π_1 there. We therefore define the *precision gap* $\Delta_k(s)$ of policy k at state s as how much π_k diverges from π_1 starting from s, with $\Delta_1(s) = 0$. We consider two estimators, in increasing order of cost and fidelity.

Myopic action divergence. Running all K policies on the same input s gives their action distributions directly. For OpenVLA, an action is a sequence of $J = 7$ discrete tokens $a = (a^1, \ldots, a^J)$, so by the chain rule the KL divergence decomposes exactly over tokens:

$$\hat{\Delta}_k^{\mathrm{KL}}(s) = \sum_{j=1}^{J} \mathbb{E}_{a^{<j} \sim \pi_1}\Big[D_{\mathrm{KL}}\big(\pi_1(a^j \mid s, a^{<j}) \,\|\, \pi_k(a^j \mid s, a^{<j})\big)\Big], \tag{3}$$

which we evaluate by teacher-forcing both models on the tokens decoded by π_1. For policies with continuous actions such as π_0, we use the expected squared distance between actions, $\hat{\Delta}_k^{\ell_2}(s) = \mathbb{E}\big[\|a^{(1)} - a^{(k)}\|_2^2\big]$, with shared sampling noise. This estimator costs one forward pass per policy but ignores whether a deviation actually affects the outcome: the policy can recover from a small deviation in free space but not from a missed grasp.

Multi-step rollout divergence. To capture whether a deviation compounds or is corrected, we reset the simulator to s, execute one action from π_k, and then follow π_1 for H steps, comparing against a rollout that follows π_1 throughout with the same random seed:

$$\hat{\Delta}_k^{H}(s) = \frac{1}{M} \sum_{m=1}^{M} d\big(x_{m,H}^{(k)}(s),\, x_{m,H}^{(1)}(s)\big), \tag{4}$$

where $x_{m,H}^{(k)}(s)$ is the end-effector and object state after H steps in the m-th rollout and d is a distance on that state. Running the rollouts to the end of the episode ($H = T - t$) and comparing success instead of state gives a direct Monte Carlo estimate of the drop in success probability, $\hat{\Delta}_k^{\mathrm{succ}}(s) = \frac{1}{M}\sum_m \big(\mathrm{succ}(\tau_m^{(1)}) - \mathrm{succ}(\tau_m^{(k)})\big)$, at the highest cost. These estimators require resetting the simulator to arbitrary states, which LIBERO supports.

2.3 Learning the Router

We train the router offline by supervised learning, using the estimated precision gaps as labels (Algorithm 1). Each iteration has a data generation phase, which rolls out a policy and labels the states it visits, and a learning phase, which fits the router to all labeled states so far. The first iteration rolls out π_1; later iterations roll out the routed policy π_ϕ and aggregate its states into the dataset (DAgger-style). This matters because routing decisions change which states are reached, so the router must be trained on the states it will actually encounter.

Algorithm 1 Offline router learning

Require: Quantized policies $\{\pi_k\}_{k=1}^K$ with costs $\{c_k\}_{k=1}^K$, simulator, gap estimator $\hat{\Delta}$, episodes per iteration N, iterations I

$\mathcal{D} \leftarrow \emptyset$
for $i = 0, 1, \ldots, I$ **do**
  $\pi_{\text{roll}} \leftarrow \pi_1$ **if** $i = 0$ **else** π_ϕ
  Data generation phase
  for $n = 1, \ldots, N$ **do**
    Roll out π_{roll} and record visited states $s_0, \ldots, s_{T-1}$
    for each visited state s_t **do**
      Run all K policies on s_t and compute $\{\hat{\Delta}_k(s_t)\}_{k=1}^K$
      Compute label $y(s_t)$ and weight $w(s_t)$ (Eqs. (5)–(6))
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(z_t, y(s_t), w(s_t))\}$
    end for
  end for
  Learning phase
  Train g_ϕ on $\mathcal{D}$ by minimizing $\mathcal{L}_{\text{CE}}(\phi)$ (Eq. (7))
end for
return g_ϕ

Learning loss. Each state receives a hard label, either the precision with the best tradeoff between gap and cost, or the lowest precision whose gap is within a tolerance ϵ:

$$y(s) = \arg\min_k \left[\hat{\Delta}_k(s) + \lambda c_k\right] \qquad \text{or} \qquad y(s) = \max\{k : \hat{\Delta}_k(s) \le \epsilon\}. \tag{5}$$

Misrouting is most costly at critical states, where the low-precision policies diverge most from π_1. We therefore weight each state by

$$w(s) = 1 + \alpha \max_k \hat{\Delta}_k(s), \tag{6}$$

and train the router with a weighted cross-entropy loss:

$$\mathcal{L}_{\text{CE}}(\phi) = -\,\mathbb{E}_{(z,y,w)\sim\mathcal{D}}\big[\, w \,\log g_\phi(y \mid z)\,\big]. \tag{7}$$

3 Experiments

4 Evaluation

We will evaluate the resulting system on:

- task success rate,
- inference latency,
- energy usage, and
- roofline analysis.

Research assignments

Brianna:
Lester:
Shresth:
Hang:

Classification model to switch precision per step

We want to build a classification model that determines whether to switch precision at each action step during deployment.

4.1 Classifier features

- The classifier itself has tiny compututation.The computationally expensive part is obtaining the features. If a feature requires high computation, it costs more than the savings from low precision.
- So we prefer pre-computed features or features that come from previous steps and ideally overlapped with the GPU's image-processing phase (as DyQ-VLA does on the CPU).

4.1.1 Candidate features (table 1)

4.1.2 Expensive features

- **Separate vision encoder or extra image processing**: adds a second expensive pass.
- **Hidden-state similarity across steps** (as in ElegantVLA's first-layer check): this is only cheap if the states are already cached; check actual latency.
- **Low-precision proposed action for the current step** (cascade design): fine as a feature, but every rejection costs a second pass. Only pays off if rejections stay below roughly $1 -$ (low-precision cost / full-precision cost), i.e. about 33% at a 1.5× speedup.

4.2 What to quantize: activations, weights, or both

A model has two kinds of numbers we could store with less bits:

- **Weights**: what the model learned in training. They are stored permanently and read again at every step.
- **Activations**: the working numbers the model computes from the current camera image and instruction. They are created fresh at each step and then thrown away.

Feature	Source	Cost	Notes
Previous action(s): translation xyz, rotation, gripper command	Model's own past outputs	Free	Short history window (e.g. last 5–10 steps)
Action changes between steps (translational and rotational change)	Computed from action history	Negligible	Captures sudden twists and slowdowns
Motion Fineness and Angular Jerk (from DyQ-VLA paper)	Computed from action history	Negligible	Useful baseline features; lets us compare against the heuristic directly
Joint positions and velocities	Robot proprioception	Free	Already read every control step
End-effector pose and speed	Proprioception / forward kinematics	Negligible	
Gripper state (open/closed, width)	Proprioception	Free	Strong cue for grasp and release phases
Token confidence: entropy and top-1 vs top-2 margin per action token	Previous step's output distribution	Free	OpenVLA-style discrete action tokens give these for free
Episode progress (step t / expected max steps)	Step counter	Free	ElegantVLA uses this as a coarse temporal cue
Previous precision decisions	Dispatcher state	Free	Helps the classifier learn smooth switching
Force/torque or contact signals	Wrist sensor, if available	Free (if hardware has it)	Possibly the most direct contact cue; not available on every robot
Task instruction embedding	Computed once per episode	Negligible per step	Lets the classifier learn task-specific behaviour

Table 1: Candidate classifier features.

An OpenVLA step has two phases, and each is slowed down by something different:

- **Image + instruction processing**: hundreds of tokens are handled at once. **The arithmetic is the slow part.**
- **Action generation**: 7 action tokens are generated one at a time, and each one reads every weight. **Reading weights from memory is the slow part**. Rough estimate on an A100: ∼7–9 ms per token at FP16, so ∼50–60 ms for all 7. At 4-bit weights it's about a quarter of that.

This matters because lowering weight precision mostly speeds up action generation, while lowering activation precision mostly speeds up image processing.

4.2.1 Option 1: Switch activation precision (DyQ-VLA's approach)

Weights stay fixed (DyQ-VLA keeps them at 4 bits), and each step picks 2, 4 or 8 bits, or full precision, for the activations.

Benefits

- **Switching precision costs nothing**: activations are recomputed every step anyway, so there's nothing extra to store or load.

- It has already been shown to work. DyQ-VLA gets ∼1.5× speedup while keeping 99.5% of performance.
- It speeds up the image phase, if the hardware has fast low-precision math.

Problems

- **Almost no memory saving, since activations are temporary**. DyQ-VLA's memory savings come from its fixed 4-bit weights, not from the switching.
- **Limited speedup in action generation, which is slowed by reading weights rather than by arithmetic**. This is likely why the speedup tops out around 1.5×.
- It needs custom low-precision math code. DyQ-VLA's switching first added 16.8 ms per step until they wrote custom kernels and moved the decision logic onto the CPU.
- Activations can be hard to round accurately. In language models they often contain a few extreme values that make them harder to round than weights.

4.2.2 Option 2: Switch weight precision (nested weights)

Store the weights once at the highest precision we want (e.g., 8 bits) in a "nested" layout, where the lower-precision versions are contained inside the higher-precision one. Each weight's first few bits give a rough value, and more bits refine it, like reading more digits of a price. The bits are stored grouped by position (the first bit of every weight together, then the second bit of every weight, and so on), so running at lower precision just means reading fewer groups. Prior work: Any-Precision LLM (ICML 2024) and Matryoshka Quantization (ICML 2025).

Related work (details in "Related work" below)

- **Fixed 4-bit weights already work for OpenVLA.** QAIL and SQIL fine-tune OpenVLA to 4-bit weights and lose about 1 point of success, while running 2.5× faster on a Jetson AGX Orin (15.2 → 4.0 GB). QVLA and Mix-QVLA show that 4-bit weights with full-precision activations match the full-precision success rate without fine-tuning (76.5% and 76.6% vs. 76.5%). So an 8-bit top level is probably unnecessary. **We could store 4 bits as the top level and drop to 2–3 bits on easy steps, using about the same memory as DyQ-VLA.**
- **MotionVQ** is the closest prior work. It switches between a coarse and a fine set of weights every step based on how much the arm moved. But it stores two separate sets rather than nested ones, uses a fixed motion threshold, and reports speed only on a simulated custom chip. According to the summary, it uses the coarse weights during *low*-motion steps, the opposite of DyQ-VLA's assumption, so check the paper.
- **MoBiQuant** (language models, not robotics) uses nested weights and picks a precision for each token with a router. It's the closest example of nested weights with runtime switching.
- **BitVLA** shows weights can go as low as $-1/0/1$ if the model is trained that way from scratch, so very low weight precision isn't hopeless for VLAs.

Benefits

- It targets the main bottleneck in action generation, so **it could beat Option 1's speedup.**

- **Only one copy of the model is stored**, and switching just means reading more or fewer bit groups.
- **It makes a cascade much cheaper**. Rerunning only the action tokens at higher precision costs just the extra bit groups, not a second full pass, so the ∼33% break-even estimate above should be redone.

Problems

- **The highest available precision is whatever we store**, so there's no FP16 unless we pay the memory for it. We need to check that 8-bit weights alone keep OpenVLA's success rate.
- It uses more memory than DyQ-VLA: ∼7 GB at 8 bits vs. their 4.7 GB, though still far below FP16's 15.2 GB.
- Simply chopping off bits could give a worse low-bit model than one purposely made for that bit-width. This hasn't been tested on VLAs.
- It needs special code that reads the bit groups. Any-Precision LLM released code for LLaMA-style models (OpenVLA's backbone is Llama 2), but it may need adapting.
- **It does little for the image phase**, which is slowed by arithmetic.
- Very low weight precision (2-bit) may lose a lot of accuracy and may only be usable during the easiest parts of a task.

Simpler alternatives that don't work:

- **Keep a separate copy at each precision on the GPU.** 2 + 4 + 8 + 16 bits is ∼30 bits per weight, which is more memory than FP16 alone.
- **Load copies from the CPU when needed.** Moving a 4-bit 7B model (∼3.5 GB) over PCIe takes ∼140 ms, longer than a whole control step.

4.2.3 Option 3: Switch both

Use weight precision for the action-generation phase and activation precision for the image phase, so each targets the phase where it's the bottleneck.

Related work (details in "Related work" below)

- **Fixed weights + activations at 4 bits already keeps around the same success rate.** QVLA (OpenVLA: 76.0% vs. 76.5%, 4.3 GB, 1.47×), Mix-QVLA (OpenVLA: 76.3% vs. 76.5%, 4.0 GB, 1.52×), QuantVLA, HoloQ-VLA and FoldQuantVLA all keep success within about a point of full precision at 4-bit weights and 4- or 8-bit activations. **This is the bar Option 3 has to beat**. Matching static 4-bit isn't enough: **the gain must come from going *below* 4 bits during easy steps.**
- **No paper switches both per step.** DyQ-VLA switches only activations and MotionVQ only weights.
- **FoldQuantVLA** found that keeping a few sensitive layers at 8 bits raised real-robot success from 80.0% to 92.5%. **Fixed per-layer exceptions like this could be combined with per-step switching for the remaining layers.**

- **BitVLA** ($-1/0/1$ weights, 8-bit activations, 1.4 GB, 96.0% on LIBERO) shows how far fixed precision for both can go when the model is trained for it.

Benefits

- **The largest potential speedup, since both phases get faster.**
- **The two can be controlled separately.** For example, the image phase could always run with low-precision activations while the classifier only chooses weight precision for the action tokens, which keeps the decision simple.

Problems

- **Errors from both sources add up**, and the combined effect on action quality is untested.
- **The classifier has more choices** (pairs of settings), which means more labels and rollouts and harder calibration. We could limit it to a few preset combinations.
- **It needs the most engineering**: two sets of low-precision code and two sources of switching overhead.

4.2.4 Open questions to test

- Does OpenVLA keep its success rate with 8-bit weights and full-precision activations? This sets the highest precision for Option 2.
- At each bit-width, **which hurts OpenVLA's action accuracy more: rounding weights or rounding activations?**
- **On our hardware, how is the wall-clock time per step split between image processing and action generation? This decides which option has more to gain.**
- How much does the image phase's precision affect action quality?

4.3 Related work

4.3.1 Fixed precision, weights only

- **QAIL** (Quantization-Aware Imitation Learning): fine-tunes OpenVLA to 4-bit weights. 73.1% vs. 74.0% on LIBERO, with 2.5× speedup and energy savings on a Jetson AGX Orin.
- **SQIL** (Saliency-Aware Quantized Imitation Learning): same setting as QAIL (OpenVLA and $\pi 0$, 4-bit weights). 73.2% vs. 73.8% on LIBERO, 2.5× faster on Jetson AGX Orin, 15.2 → 4.0 GB. During fine-tuning it weights the robot states where small action errors matter most (top 20% by perturbation sensitivity) more heavily. That's a fixed-precision cousin of our per-step sensitivity labels.

4.3.2 Fixed precision, weights and activations

- **QVLA** (ICLR 2026): each output channel gets its own bit-width from {0, 2, 4, 8, 16}, set once offline. OpenVLA-OFT at W4A4: 96.0% vs. 97.1%, 4.5 GB, 1.49×. OpenVLA at W4A4: 76.0% vs. 76.5%, 4.3 GB, 1.47×. OpenVLA at W4A16 matches full precision. This is the main baseline DyQ-VLA compares against.
- **Mix-QVLA**: one bit-width per layer from {2, 4, 8, 16}, chosen offline by optimization. Fixed at deployment: it "does not perform timestep-wise bit switching during deployment." OpenVLA at W4A4: 76.3% vs. 76.5%, 4.0 GB, 1.52×. W4A16: 76.6%.
- **QuantVLA**: W4A8 on π0.5 and GR00T N1.5, no training. π0.5: 97.6% vs. 97.1%, 4.27 → 1.28 GB.
- **HoloQ-VLA**: uniform W4A4 on π0.5 and GR00T N1.5, including the diffusion action head. π0.5: 98.0% vs. 97.1%.
- **FoldQuantVLA**: W4A4 on GR00T and π0.5, with 1.2–1.5× speedups on real hardware, including a Jetson AGX Orin. Keeping a few sensitive layers at W8A8 raised real-robot success from 80.0% to 92.5%.
- **BitVLA**: a new 3B VLA trained with $-1/0/1$ weights and 8-bit activations. 1.4 GB, 96.0% on LIBERO. It's trained from scratch rather than being a compressed existing model.

4.3.3 Per-step (timestep-wise) switching

- **DyQ-VLA**: weights fixed at 4 bits. Activation precision switches every step (2/4/8 bits or full precision) based on a motion score.
- **MotionVQ** (arXiv 2607.24148, July 2026): weights only, using vector quantization. It keeps a coarse and a fine set of weights and switches between them every step based on motion in the previous step. Tested on five VLAs (OpenVLA, OpenVLA-OFT, RDT, π0, GR00T), with ∼2.5% success loss. Speedups come from a simulated custom chip, not a real GPU. I've only read a summary, not the paper.

4.3.4 Outside robotics: nested weights

- **Any-Precision LLM** (ICML 2024) and **Matryoshka Quantization** (ICML 2025): store one copy of the weights that can run at several precisions.
- **MoBiQuant** (2026): nested weights, with a router that picks the precision for each token at inference. Up to 1.34× faster than earlier any-precision methods. The paper page doesn't say how the router is trained.

4.3.5 Who decides when to switch?

None of the robotics papers above uses a supervised (trained) model to decide when to switch precision.

- DyQ-VLA uses a hand-designed score (Motion Fineness + Angular Jerk) with fixed cutoffs.
- MotionVQ calls its switch a "state predictor," but the summary I read describes a fixed motion threshold (0.8). Check the paper to confirm nothing in it is learned.

- MoBiQuant has a per-token router, but in language models, and its training wasn't checked.

Gap for our project: a learned (supervised) classifier that decides per step, possibly combined with nested weights. This field moves fast (several of these papers are from the last few months), so search again before committing.